

Nombre:

DNI:

Primer control de teoría

Responde brevemente a las siguientes preguntas justificando tus respuestas. Una respuesta sin justificar no será dada como válida. Todas las preguntas tienen la misma puntuación.

Su búsqueda del gato She Thao le había llevado hasta un suburbio de Londres donde se alojaba en un humilde hostel para huir de cualquier mirada indiscreta. O al menos eso es lo que Baka Baka creía. Después de un día agotador en el que reconoció la zona buscando pistas sobre el paradero del gato, al entrar en su habitación, vio sobre su cama una pequeña caja de cartón que alguien había depositado cuando él estaba fuera. Se acercó con sorpresa a la caja y al abrirla, de ella salió un espeso humo negro. Cayó desmayado.

Horas después, se despertó en una habitación totalmente blanca y fuertemente iluminada en la que, sobre una mesa también blanca, reposaba un portátil, una placa base de un ordenador y un libro bastante grueso.

Voz: “Sabemos que usted es el famoso Baka Baka. Queremos que implemente un sistema operativo que funcione sobre el procesador de la placa base que tiene delante de usted. Para ello, le hemos proporcionado el manual de funcionamiento del procesador y un ordenador portátil para que pueda trabajar. Tiene 72 horas o morirá.”

Baka Baka se fijó que, en una esquina de la habitación, había una cámara de seguridad y un altavoz de donde, sin duda, provenía aquella voz. “¿Quién es? ¿Quién está hablando?”. Nadie contestó. En la pared opuesta se encendió un contador: “71:59:59” ... “71:59:58” ... “71:59:57” ... La amenaza parecía seria, aunque él no se dejaría amedrentar.

Después de lo que le parecieron horas, su curiosidad le empezó a traicionar. Se dio cuenta de que la placa base tenía un procesador que no había visto nunca y que no se parecía a ninguna tecnología que él conociese. En el libro ponía: “Руководство по программированию операционных систем для процессора Бизон-11”. “¡Ruso!”, pensó, y, antes de darse cuenta, había empezado a oírla.

65:35:23 ... 65:35:22 ...

Cuando más enfrascado estaba en la lectura del manual del procesador, un hombre de mediana edad, impecablemente trajeado y que desprendía cierto olor a ginebra y vodka, irrumpió en la habitación. Con un inglés perfecto, le preguntó: “Señor Baka Baka, ¿Cómo va?”. “Es muy interesante”, contestó Baka Baka con una voz profunda haciéndose el interesante, “es básicamente un procesador compatible con un x86 pero con la instrucción SYSSWAP en vez de int, sysenter o syscall para invocar al sistema operativo”.

Voz en off: la instrucción SYSSWAP fuerza un cambio de nivel de privilegios. En el caso de que el modo de privilegios actual sea el 3 (mínimo nivel de privilegios), cambia al 0 (máximo nivel de privilegios). Si el nivel actual es el 0, cambia al 3. En cualquier caso, accede a la dirección física 0x4000 donde tiene que haber una estructura con los siguientes campos:

```
struct sysswap_info
{
    unsigned long new_esp;
    unsigned long new_eip;
    unsigned long new_cr3;
};
```

SYSSWAP siempre **intercambia** los valores actuales de esp, eip y cr3 por los de esa estructura. Por ejemplo, para el registro esp, SYSSWAP haría algo parecido a:

```
unsigned long temp = esp;
esp = ((struct sysswap_info*)0x4000)->new_esp;
((struct sysswap_info*)0x4000)->new_esp = temp;
```

El comportamiento es similar para los registros new_eip y new_cr3. El valor del registro eip es la dirección de la siguiente instrucción después de SYSSWAP. SYSSWAP siempre hace un flush del TLB. **Para simplificar, no nos preocuparemos del cambio de nivel de privilegios con interrupciones hardware ni excepciones.**

“Por tanto”, continuó Baka Baka, “las tablas de páginas en modo usuario y modo sistema son diferentes. La tabla de páginas que se utiliza en modo usuario, USER_TP, solamente tiene la traducción de direcciones lógicas a direcciones físicas para el código, datos y pila(s) de usuario. La tabla de páginas que se utiliza en modo sistema, SYS_TP, solo contiene las traducciones de direcciones lógicas referentes al kernel. Por lo demás, todo es igual: las tablas de páginas son de dos niveles, el tamaño de página es de 4KB...”. “Muy interesante” le dijo el desconocido mientras dejaba un sándwich y una botella de agua sobre la mesa. Acto seguido, le regaló una mirada de desprecio a Baka Baka y se fue.

Baka Baka encendió el ordenador y se puso a trabajar. Seguramente, cualquier psiquiatra competente diría que Baka Baka sufre un caso claro de esquizofrenia, pero también es cierto que esa vocecilla interior (que eres tú) que oía en los momentos de más tensión, le había ayudado en incontables ocasiones. Empezó por la inicialización del sistema.

Pregunta 1

Baka Baka: ¿Qué valores tengo que poner en cada uno de los campos de la estructura sysswap_info?

Hay 2 opciones:

1.- Si se vuelve con un mecanismo parecido a return_gate:

new_esp : dirección lógica de la pila de sistema

new_eip: dirección lógica del handler de las llamadas al sistema

new_cr3: dirección física de la tabla de páginas del sistema operativo

2.- Si se vuelve con SYSSWAP. Hay que asegurar que en cr3 esté la tabla de páginas de sistema y se esté utilizando la pila de sistema del proceso Init:

new_esp: dirección lógica de la pila de usuario

new_eip: dirección lógica de la primera instrucción a ejecutar en modo usuario

new_cr3: dirección física de la tabla de páginas de usuario.

Para volver a usuario se tiene que hacer:

SYSSWAP

jmp systemcall_handler

Pregunta 2

Baka Baka: pero la inicialización de esta estructura, no puedo hacerla en cualquier momento. ¿En qué punto del código de boot es seguro poner el código de inicialización de la estructura sysswap_info?

La tabla de páginas del sistema operativo y el proceso init tienen que crearse antes de inicializar la estructura sysswap_info

Además, Baka Baka sabía que el contenido de esa estructura, seguramente, tendría que ser modificado más adelante en el código del sistema operativo.

SOA2 (20/04/2026)

Nombre:

DNI:

Pregunta 3

Baka Baka: tengo que poder acceder desde cualquier punto del código de sistema, y en cualquier momento, a la estructura `sysswap_info`. ¿Cómo preparo la memoria para conseguirlo?

Se tiene que mapear la dirección física `0x4000` a una dirección lógica. Para simplificar, mapearemos la dirección física `0x4000` a la lógica `0x4000`. Además, marcaremos la página física `0x4` como ocupada.

```
reserve_frame(4);
set_ss_pag(SYS_TP, 4, 4);
```

Pregunta 4

Baka Baka: quiero probar una cosa: cuando creo las tablas de páginas para el sistema operativo y para los procesos, ¿se podrían implementar de tal forma que las direcciones lógicas del código de sistema y las del código de usuario fuesen las mismas?

Si que se puede hacer ya que las tablas de páginas de modo sistema y de modo usuario son diferentes y se cambian cuando se cambia el modo. Por tanto, el código de sistema y el de usuario estarían en direcciones físicas diferentes, pero podrían mapearse en las mismas direcciones lógicas.

Pregunta 5

Baka Baka: voceilla, dime qué código tengo que poner para crear la tabla de páginas que se utilizará en modo sistema teniendo en cuenta que el sistema operativo solamente ocupará 200 KBs de memoria, que la dirección lógica inicial del sistema será la `0x10000` y que existe una función llamada `set_system_pages` que inicializa todos los mapeos de direcciones lógicas a físicas para el kernel.

Solamente se necesitará un segmento de la tabla de páginas que se mapeará en la entrada 0 del directorio.

```
int dir = alloc_frame();
int sys = alloc_frame();

set_ss_pag(dir, 0, sys);
set_ss_pag(sys, dir, dir);
set_ss_pag(sys, sys, sys);
set_system_pages(dir << 12);
```

Una vez comprobado que el código de boot funcionaba, toca implementar los wrappers.

44:16:00 ... 44:15:59 ...

Pregunta 6

Baka Baka: sé que existen 3 formas de pasar los parámetros de los wrappers al sistema operativo, pero en este caso, me da la sensación de que solo puedo utilizar una de ellas. ¿Estoy en lo cierto?

Si, debido a que no se puede acceder a la memoria del usuario desde modo sistema, solamente se pueden pasar los parámetros por registros.

37:22:44 ... 37:22:43 ...

El hombre trajeado entró otra vez en la habitación, pero esta vez llevaba un pastel de carne en vez de un sándwich. “¿Algún progreso significativo?” preguntó. Baka baka, a quien le empezaba a hacer mella el cansancio, contestó: “Llevo horas depurando el handler del sistema porque no funciona siempre y

no soy capaz de saber por qué”. “M, este hombre está loco”, susurró el desconocido a su solapa y se marchó.

El handler, sin comprobación de errores, que escribió Baka Baka era:

```
ENTRY(syscall_handler)
    SAVE_ALL
    call *syscall_table(, %eax, 4)
    movl 0x18(%esp), %eax
    RESTORE_ALL
    SYSSWAP
```

Pregunta 7

Baka Baka: sospecho que se debe a la pila de sistema. ¿Qué contiene la pila de sistema antes de ejecutar SAVE_ALL?

Nada, SYSSWAP no guarda nada en la pila de sistema.

Pregunta 8

Baka Baka: ayúdame vocecilla, ¿por qué funciona este handler solamente para la primera llamada al sistema, pero no para las siguientes?

Cuando SYSSWAP se ejecuta, guarda en ((struct sysswap_info*)0x4000)->new_eip la dirección de la siguiente instrucción después de SYSSWAP. Esto hace que la siguiente llamada al sistema, salte a la instrucción después de SYSSWAP en vez de a la primera instrucción del handler.

Pregunta 9

Baka Baka: ¡ahora lo entiendo! Pero, ¿Cómo lo soluciono?

Para solucionarlo, después de SYSSWAP se tiene poner:

jmp syscall_handler

Pregunta 10

Baka Baka: ahora que pienso... ¿necesito modificar el handler de las llamadas al sistema para guardar el contexto hardware como se hacía con sysenter?

No es necesario ya que se está guardando en sysswap_info. Para ser detallistas, a lo mejor se tendría que guardar cs, ss y la palabra de estado del procesador pero si no se hace, no pasa nada.

Baka Baka probó el código de los wrappers y del handler, implementando la llamada al sistema getpid. Funcionaba. Pero se dio cuenta de que, aunque el procesador funcionaba exactamente igual que un x86, excepto por la instrucción SYSSWAP, el código tardaba más tiempo en ejecutarse.

Pregunta 11

Baka Baka: ¿Por qué va más lento ejecutar una llamada al sistema?

Porque cada vez que se hace un cambio de modo, se fuerza un flush del TLB lo que hace que se produzcan muchos fallos de TLB.

20:59:59 ... 20:59:58 ...

Baka Baka necesitaba descansar. Llevaba mucho tiempo sin dormir, pero no se lo podía permitir: el tiempo se acababa y con él, posiblemente, su vida. Además, quedaba lo más difícil: task_switch y sys_fork.

SOA2 (20/04/2026)

Nombre:

DNI:

Voz en off: recuerda que estamos en teoría y aquí no se ha explicado que task_switch llame a inner_task_switch... eso es del laboratorio.

Pregunta 12

Dibuja, de la forma más detallada posible, el contenido de la pila de sistema hasta antes de ejecutar la primera instrucción en alto nivel de task_switch.

Del bottom al top de la pila:

GS, FS, ES, DS, EAX, EBP, EDI, ESI, EDX, ECX, EBX, @ret handler, ... , new, @ret, EBP

Pregunta 13

Baka Baka: a parte del valor de ebp actual, ¿se tiene que guardar alguna otra información dentro de task_switch?

Si. Se tienen que guardar los valores actuales de la estructura sysswap_info para el proceso actual para luego restaurarlos cuando vuelva a entrar en la cpu

Pregunta 14

Baka Baka: entiendo. ¿Y dónde guardo esa información?

Dentro del struct task_struct del proceso habría que añadir una struct sysswap_info y guardar los valores allí.

Pregunta 15

Baka Baka: entonces, ¿cómo es el código resultante del task_switch en pseudo-C-ensamblador?

```
void task_switch(struct task_struct *new)
{
    current()->sysswap.new_eip = ((struct sysswap_info*)0x4000)->new_eip;
    current()->sysswap.new_esp = ((struct sysswap_info*)0x4000)->new_esp;
    current()->sysswap.new_cr3 = ((struct sysswap_info*)0x4000)->new_cr3;
    ((struct sysswap_info*)0x4000)->new_eip = new->sysswap.new_eip;
    ((struct sysswap_info*)0x4000)->new_esp = new->sysswap.new_esp;
    ((struct sysswap_info*)0x4000)->new_cr3 = new->sysswap.new_cr3;
    tss.esp0 = &((union task_union*)new)->stack[1024]; // for int hw & exceptions
    current()->kernel_esp = ebp;
    esp = new->kernel_esp;
    pop ebp;
    ret;
}
```

Pregunta 16

Baka Baka: puede que me equivoque, pero ¿la función task_switch no es mucho más rápida en este procesador?

Si ya que ahora no se toca el registro cr3 y por tanto no se hace flush del tlb.

07:07:07 ... 07:07:06 ...

Las horas de trabajo se acumulaban y eran solamente interrumpidas por las idas y venidas del desconocido que traía comida y que hablaba a su solapa. Baka baka se sentía exhausto, pero sabía que estaba acabando.

Pregunta 17

Baka Baka: supongo que tendré que incluir, en sys_fork, código para inicializar la estructura sysswap_info para el proceso hijo, ¿no?

Si, se tiene que preparar la estructura sysswap_info del hijo para que utilice la tabla de páginas del proceso hijo:

```
struct task_struct *child; // Task struct del proceso hijo
child->sysswap.new_eip = ((struct sysswap_info*)0x4000)->new_eip;
child->sysswap.new_esp = ((struct sysswap_info*)0x4000)->new_esp;
child->sysswap.new_cr3 = &directorio_hijo;
```

Pregunta 18

Baka Baka: por otro lado, creo que SYSSWAP no afecta a cómo se tiene que copiar la zona de memoria de usuario del proceso padre al proceso hijo. ¿Es esto cierto?

Si... en este caso, como el sistema no tiene acceso a la memoria de usuario del proceso, dentro del bucle de copia, se tiene que mapear tanto la página física del proceso padre como la del hijo en la tabla de páginas del sistema operativo para poder acceder a su contenido.

Pregunta 19

Escribe el código del bucle para copiar la zona de datos del padre a la zona de datos del hijo suponiendo que la tabla de páginas del proceso padre, del hijo y del sistema se llaman TPp, TPh y TPs respectivamente. La zona de datos de un proceso empieza en la página lógica PAG_LOG_INIT_DATA y tiene PAG_NUM_DATA páginas. Puedes suponer que las nuevas páginas del proceso hijo están mapeadas ya en la TPh donde corresponde. En la TPs, a partir de la PAG_FREE_KERNEL hay suficientes páginas lógicas libres.

```
for (int pag = 0; pag < PAG_NUM_DATA; pag++)
{
    set_ss_pag(TPs, PAG_FREE_KERNEL+2*pag, get_frame(TPp, PAG_LOG_INIT_DATA + pag));
    set_ss_pag(TPs, PAG_FREE_KERNEL+2*pag+1, get_frame(TPh, PAG_LOG_INIT_DATA + pag));
    copy_data(PAG_FREE_KERNEL + 2 * pag, PAG_FREE_KERNEL + 2 * pag + 1, PAGE_SIZE);
    del_ss_pag(TPs, PAG_FREE_KERNEL + 2 * pag);
    del_ss_pag(TPs, PAG_FREE_KERNEL + 2 * pag + 1);
}
```

Pregunta 20

Baka Baka: supongo que también tendré que hacer flush del TLB después de acabar la copia, ¿correcto?

No... no hace falta poner el flush ya que se hará cuando se vuelva a modo usuario.

00:00:06 ... 00:00:05 ...

“Funciona!!”. Baka baka no cabía en sí de la alegría. En ese momento, el hombre trajeado irrumpió en la habitación. Miró con desprecio a Baka baka y se acercó a la mesa. Sacó un pequeño pendrive del bolsillo y lo conectó a la placa base. No pasó nada. “Obviamente, eso no funcionará”, dijo Baka Baka. El desconocido se giró y le miró inquisidoramente. Baka Baka le respondió: “Porque aún no he implementado un sistema de ficheros. Eso no toca ahora”. El hombre, lleno de ira, le pegó un puñetazo a Baka Baka tirándole al suelo. Baka Baka asustado, balbuceó: “Pero... ¿Por qué? ¿Quién eres tú?”. El hombre sacó una pistola Walther PPK del bolsillo y le apuntó directamente a la cabeza mientras, con una voz solemne, le dijo: “Es verdad, no nos han presentado. Mi nombre es Bond, James Bond”.

(continuará)

SOA2 (20/04/2026)

Nombre:

DNI:

El sistema dispone de las siguientes rutinas:

- ***struct task_struct *current()***: Retorna la *task_struct* del proceso actual.
- ***int alloc_frame()***: Reserva un frame de memoria física.
- ***int reserve_frame(int frame)***: Marca el frame físico *frame* como ocupado.
- ***void free_frame (int frame)***: Libera un frame de memoria.
- ***page_table_entry * get_PT(struct task_struct *t)***: Retorna la tabla de páginas del proceso *t*.
- ***page_table_entry * get_DIR(struct task_struct *t)***: Retorna el directorio de páginas del proceso *t*.
- ***set_CR3 (page_table_entry * dir)***: Sobreescribe el registro CR3 con el nuevo directorio de páginas *dir*, provoca una invalidación de la TLB.
- ***int get_frame (page_table_entry *pt, int logical_page)***: Retorna el frame asignado a una página lógica en la tabla de páginas de un proceso determinado.
- ***int set_ss_page (page_table_entry *pt, int logical_page, int frame)***: Asigna un *frame* a una página lógica de la tabla de páginas *pt*.
- ***int del_ss_page (page_table_entry *pt, int logical_page)***: Borra una entrada de la tabla de páginas.